

Project Improvement Roadmap

Multi-Agent Infrastructure Compliance Engine — Accuracy & Reliability Upgrades

*Honest assessment first: your project is **architecturally sound** but has several **silent accuracy leaks**. Below I've ranked every improvement by impact-per-effort. If you only have one weekend, do Tier 1. If you have a month, do Tiers 1–2. The rest are long-term vision.*

TIER 1 — Critical Accuracy Fixes (Do These First)

These are bugs that are **actively reducing your accuracy right now**. Each one is a 30-minute fix with huge payoff.

Fix 1: The Silent Knowledge Base Truncation 🐞

The bug — In `agent1_forest_compliance.py`, line 324:

```
SOURCE C — KNOWLEDGE BASE (Official Forest Compliance Rules)
{rules_text[:3500]}
```

That `[:3500]` is **silently cutting off your rules**. If you have 20 rules in your JSON files, only the first ~10 reach the LLM. The agent then audits against a half-blind ruleset and you never know which rules it skipped.

Why it's there — you were worried about token limits. But GPT-4o has a 128K context window. 3,500 characters is using <3% of it.

The fix — Replace with smart truncation only if absolutely needed:

```
# Old:
SOURCE C — KNOWLEDGE BASE
{rules_text[:3500]}

# New:
SOURCE C — KNOWLEDGE BASE (Official Forest Compliance Rules)
{rules_text if len(rules_text) < 60000 else rules_text[:60000] + "\n[Note: KB truncat
ed at 60K chars]}"
```

Accuracy impact: Huge. You might find your project was missing half your own compliance rules.

Fix 2: Temperature Should Be 0 for Compliance Audits 🎯

The bug — Every agent uses `temperature=0.2`. For compliance work, you want **zero variance**. The same DPR should produce the same audit every time. With 0.2, two runs can produce slightly different findings, which is unacceptable for legal/regulatory work.

The fix — In all four agent files:

```
# Change this:
self.llm = ChatOpenAI(model="gpt-4o", temperature=0.2, streaming=True)

# To this:
self.llm = ChatOpenAI(model="gpt-4o", temperature=0, streaming=True, seed=42)
```

The `seed=42` parameter (OpenAI added this) makes outputs **deterministic** — same input → same output. Essential for reproducibility in audits.

Keep `temperature=0.3` only for the Hindi translation in `app.py` — there, natural fluency matters more than determinism.

Accuracy impact: Massive for repeatability. Now if a court demands you re-run the audit, you can prove you got the same answer.

Fix 3: Cross-Agent Context Sharing 🗎

The problem — Right now, all four agents run independently. Agent 3 might discover a High Court mining ban that affects RBM sourcing. Agent 4 has its own search for High Court orders. **They don't share findings**. So Agent 4 might miss the same ban Agent 3 already found.

The fix — Pass a `shared_context` dictionary that accumulates as each agent runs:

```
# In app.py or main_orchestrator.py:

shared_context = {"prior_findings": []}

# After Agent 1:
reports["agent1"] = agent1.process_dpr(dpr_pdf_path, master_project, shared_context=s
hared_context)
shared_context["prior_findings"].append(f"Agent 1 found: {reports['agent1'][:1000]}")

# After Agent 2:
reports["agent2"] = agent2.process_project(master_project, shared_context=shared_cont
ext)
shared_context["prior_findings"].append(f"Agent 2 found: {reports['agent2'][:1000]}")
# ... and so on
```

Then in each agent's prompt, inject:

```
prompt = f"""
... existing prompt ...

PRIOR AGENT FINDINGS (DO NOT REPEAT, BUT BUILD ON):
{chr(10).join(shared_context.get("prior_findings", []))}
"""
```

Accuracy impact: Eliminates contradictory findings between agents and lets later agents cite earlier findings.

Fix 4: Add a Verification / Critic Agent 🕵️

The problem — Your agents produce a report and you trust it. There's no second pair of eyes. LLMs hallucinate facts, especially URLs and dates.

The fix — Add a 5th agent that **only reviews the other agents' outputs** before showing them to the user:

```
# In app.py, after all four agents finish:

from langchain_openai import ChatOpenAI
from langchain_core.messages import HumanMessage

def critic_review(report_text: str, agent_name: str) -> str:
    """A skeptical reviewer that flags hallucinations and overconfident claims."""
    critic = ChatOpenAI(model="gpt-4o", temperature=0)
    prompt = f"""
    You are a SKEPTICAL CHIEF AUDITOR reviewing a draft report from {agent_name}.
    Your job: flag any of the following problems:

    1. URLs that look fabricated (wrong domain pattern, suspicious paths)
    2. Statistics or numbers given without a source
    3. Legal/regulatory claims without case/circular numbers
    4. Overconfident claims that should have hedges ("likely", "may")
    5. Any factual claim that does not appear traceable to a search result

    For each problem, output:
    ⚠️ [Type] – [Direct quote of suspicious claim] – [Why it's suspicious]

    If no problems, output: ✅ Report passes critical review.

    DRAFT REPORT TO REVIEW:
    {report_text}
    """
    response = critic.invoke([HumanMessage(content=prompt)])
    return response.content

# Apply to each agent's output:
for agent_key, agent_title in agent_names.items():
    original = reports[agent_key]
    critique = critic_review(original, agent_title)
    reports[agent_key] = f"{original}\n\n---\n\n## 🧐 Critical Review\n\n{critique}"
```

Accuracy impact: This is your **biggest single accuracy improvement**. Every claim gets cross-examined before reaching the user.

Fix 5: Force Structured Output with Pydantic

The problem — Your agents return free-text markdown. That makes downstream processing (the Hindi translation, the HTML export) fragile. If the LLM forgets to use the  **COMPLIANCE GAP** tag once, your parser breaks.

The fix — Use Pydantic schemas to force the LLM to fill specific fields:

```
from pydantic import BaseModel, Field
from typing import List
from langchain_openai import ChatOpenAI

class ComplianceFinding(BaseModel):
    finding_type: str = Field(description="One of: DATA_MISMATCH, COMPLIANCE_GAP, RIS
K_FLAG")
    severity: str = Field(description="One of: HIGH, MEDIUM, LOW")
    description: str
    source_url: str = Field(description="Source URL or 'N/A' if from internal docs")
    recommended_action: str

class AgentReport(BaseModel):
    agent_name: str
    project_name: str
    findings: List[ComplianceFinding]
    summary: str
    confidence_score: float = Field(ge=0, le=1)

# Then bind to your LLM:
structured_llm = self.llm.with_structured_output(AgentReport)
report = structured_llm.invoke(prompt)

# Now report.findings is a real list you can iterate, filter by severity, etc.
```

Accuracy impact: Eliminates parsing errors entirely. Every report becomes a real Python object with typed fields.



TIER 2 — High-Impact Improvements (Next 2 Weeks)

Fix 6: Replace Hardcoded JSON with a Vector Knowledge Base

Current state — Your knowledge base dumps *all* rules into the prompt. The LLM has to find the relevant ones itself, and irrelevant rules dilute attention.

Better approach — Embed each rule once, then retrieve only the top-K relevant rules per query:

```
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

# One-time setup (do this when loading rules):
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
rule_documents = [
    f"{r['title']}\n{r['summary']}\nFields: {' '.join(r['required_fields'])}"
    for r in self.rules
]
self.vectorstore = FAISS.from_texts(rule_documents, embeddings)

# Then at query time:
relevant_rules = self.vectorstore.similarity_search(
    query=f"forest compliance {project_district} {project_type}",
    k=10 # Only top 10 most relevant rules
)
```

Why this is better — The LLM now sees only the 10 most relevant rules instead of all 50, and attention is focused. Plus, FAISS is local and free — no API costs.

Accuracy impact: Big — irrelevant rules are noise that hurts focus.

Fix 7: Semantic PDF Chunking Instead of Character Splitting

The problem — `chunks = [full_text[i:i+10000] for i in range(0, len(full_text), 10000)]`

This splits the DPR every 10,000 characters **regardless of meaning**. A sentence about NPV can get cut in half. A table can be split across two chunks. A forest area mention can land at a chunk boundary and lose its surrounding context.

The fix — Use LangChain's recursive splitter that respects paragraphs and sentences:

```

from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=10000,
    chunk_overlap=1000, # 10% overlap preserves context across boundaries
    separators=["\n\n", "\n", ". ", " ", ""],
    length_function=len,
)

chunks = splitter.split_text(full_text)

```

Accuracy impact: Significantly better extraction quality. The overlap also catches information that straddles chunk boundaries.

Fix 8: Extract Tables Separately from Text

The problem — DPRs are full of cost tables, BOQ (Bill of Quantities) tables, and material specifications. `fitz.get_text()` flattens tables into garbled text where columns merge into rows. Critical data gets lost.

The fix — Use `pdfplumber` (better at tables) or `camelot-py` alongside PyMuPDF:

```

import pdfplumber

def extract_with_tables(pdf_path):
    text = ""
    tables = []
    with pdfplumber.open(pdf_path) as pdf:
        for page in pdf.pages:
            text += page.extract_text() + "\n"
            page_tables = page.extract_tables()
            for tbl in page_tables:
                # Convert table to markdown for LLM consumption
                tables.append("\n".join([" | ".join(str(c) for c in row) for row in tbl if row]))

    return text + "\n\n=== EXTRACTED TABLES ===\n\n" + "\n\n".join(tables)

```

Accuracy impact: Huge for cost validation. Now Agent 1 can actually see the cost breakdown table.

Fix 9: Add Retry Logic with Exponential Backoff

The problem — One transient OpenAI 429 error and your entire audit fails after spending money on the first three agents.

The fix — Wrap all LLM and search calls with `tenacity`:

```
from tenacity import retry, stop_after_attempt, wait_exponential, retry_if_exception_type
import openai

@retry(
    stop=stop_after_attempt(5),
    wait=wait_exponential(multiplier=1, min=4, max=60),
    retry=retry_if_exception_type((openai.RateLimitError, openai.APIError))
)
def safe_invoke(llm, messages):
    return llm.invoke(messages)
```

Then call `safe_invoke(self.llm, [...])` everywhere you currently call `self.llm.invoke(...)`. Same for search calls.

Accuracy impact: Indirect but important — failed runs produce blank reports, which is the worst kind of inaccuracy.

Fix 10: Cache Search Results

The problem — Audit the same district twice and you pay SerpAPI/Tavily twice. Plus, results that *should* be identical can vary slightly between calls.

The fix — Add a SQLite-backed cache with a TTL:

```

import hashlib
import json
import sqlite3
from datetime import datetime, timedelta

class SearchCache:
    def __init__(self, db_path="search_cache.db", ttl_days=7):
        self.conn = sqlite3.connect(db_path)
        self.conn.execute("""
            CREATE TABLE IF NOT EXISTS cache (
                key TEXT PRIMARY KEY,
                value TEXT,
                created_at TIMESTAMP
            )
        """)
        self.ttl = timedelta(days=ttl_days)

    def get(self, query):
        h = hashlib.md5(query.encode()).hexdigest()
        cur = self.conn.execute("SELECT value, created_at FROM cache WHERE key = ?",
(h,))
        row = cur.fetchone()
        if not row: return None
        created = datetime.fromisoformat(row[1])
        if datetime.now() - created > self.ttl: return None
        return row[0]

    def set(self, query, value):
        h = hashlib.md5(query.encode()).hexdigest()
        self.conn.execute(
            "INSERT OR REPLACE INTO cache VALUES (?, ?, ?)",
            (h, value, datetime.now().isoformat())
        )
        self.conn.commit()

```

Then wrap your search tools:

```

cache = SearchCache()

@tool
def search_government_tenders_serp(query: str) -> str:
    cached = cache.get(query)
    if cached: return cached + "\n[from cache]"
    result = SerpAPIWrapper().run(query)
    cache.set(query, result)
    return result

```

Accuracy impact: Indirect — but reproducibility goes up, and you save real money on repeated audits.

Fix 11: Confidence Scoring on Every Finding

The problem — Your reports say things definitively. *"Compensatory Afforestation area is missing."* But what if the LLM just didn't find it? You need explicit confidence.

The fix — Add confidence to every finding by asking the agent to grade itself:

```

# Modify the prompt:
"""
For each finding you report, you MUST include a confidence level:
- ☐ HIGH (90%+): I directly read this in the source document
- ☐ MEDIUM (60-90%): I inferred this from context or partial evidence
- ☒ LOW (<60%): I could not verify, this is my best guess

If confidence is LOW, the user will treat the finding as a 'requires manual check' item.
"""

```

Accuracy impact: Doesn't make the agent smarter, but tells the user **which findings to trust**. This is what separates a useful tool from a misleading one.



TIER 3 — Reliability & Engineering (Next Month)

Fix 12: Parallel Agent Execution

Your four agents run **sequentially** in `app.py`. Total runtime $\approx$ sum of all four. They could run in parallel since they don't depend on each other.

```
import concurrent.futures

with concurrent.futures.ThreadPoolExecutor(max_workers=4) as executor:
    futures = {
        "agent1": executor.submit(agent1.process_dpr, dpr_pdf_path, master_project),
        "agent2": executor.submit(agent2.process_project, master_project),
        "agent3": executor.submit(agent3.process_intelligence, master_project),
        "agent4": executor.submit(agent4.process_design, master_project),
    }
    reports = {k: f.result() for k, f in futures.items()}
```

⚠️ **Caveat:** Streamlit's `st.status` callbacks don't play well with threads. You'll need to use `asyncio` + `st.empty()` placeholders instead. The speedup (4× faster) is worth the rewrite if scale matters.

Fix 13: Build a Golden Dataset for Evaluation

Right now you have **no way to prove** the system is accurate. Build a small benchmark:

1. Pick 5 real DPRs where you know the correct findings (because they were already audited by humans).
2. Run the system on each one.
3. Score what percentage of human-verified findings the system catches.
4. Repeat this every time you change a prompt — see if accuracy goes up or down.

```
# tests/test_accuracy.py

GOLDEN_DPRS = [
    {
        "pdf_path": "tests/dpr_bageshwar.pdf",
        "expected_findings": [
            "Missing compensatory afforestation area",
            "NPV calculation absent",
            "KML file referenced but not attached",
        ],
    },
    # ... more cases
]

def test_agent1_recall():
    for case in GOLDEN_DPRS:
        report = agent1.process_dpr(case["pdf_path"], "Uttarakhand")
        hits = sum(1 for f in case["expected_findings"] if f.lower() in report.lower())

        recall = hits / len(case["expected_findings"])
        assert recall >= 0.8, f"Recall dropped to {recall} for {case['pdf_path']}"
```

This is what professional ML engineers call **regression testing for prompts**. Without it, "I tweaked the prompt and it feels better" is wishful thinking.

Fix 14: Persistent Storage with PostgreSQL

Streamlit `session_state` is **in-memory only**. Server restart → all reports lost. For real use:

```
import psycopg2
from datetime import datetime

def save_audit(project_name, reports):
    conn = psycopg2.connect("dbname=audits user=...")
    cur = conn.cursor()
    cur.execute(
        "INSERT INTO audits (project, agent1, agent2, agent3, agent4, created_at) VALUES (%s, %s, %s, %s, %s, %s)",
        (project_name, reports["agent1"], reports["agent2"], reports["agent3"], reports["agent4"], datetime.now())
    )
    conn.commit()
```

Then later you can: compare two audits of the same project, see how findings evolved, share specific audits via URL, and resume an audit after a crash.

Fix 15: PDF Export (Not Just HTML)

Government offices want **PDFs**, not HTMLs. Add a one-click PDF export:

```
# Add to requirements.txt: weasyprint or reportlab

from weasyprint import HTML

def html_to_pdf(html_string, output_path):
    HTML(string=html_string).write_pdf(output_path)

# In app.py download button:
pdf_bytes = HTML(string=html_data).write_pdf()
st.download_button(
    label="📄 Download as PDF",
    data=pdf_bytes,
    file_name=f"{agent_key}_report.pdf",
    mime="application/pdf"
)
```

🌟 TIER 4 — Long-Term Vision (3-6 Months)

These are "wow factor" upgrades that turn this from a project into a product.

A. Geospatial Layer (Agent 6)

Add a new agent that takes the project's **KML/coordinates** and overlays them on:

- Eco-Sensitive Zone (ESZ) shapefiles (from MoEFCC's Parivesh portal)
- Wildlife sanctuary boundaries
- Forest reserve maps

```
# pip install geopandas shapely
import geopandas as gpd
from shapely.geometry import Point

def check_eco_sensitive_zone(lat, lng):
    eszs = gpd.read_file("data/india_esz.shp")
    point = Point(lng, lat)
    intersecting = eszs[eszs.contains(point)]
    if len(intersecting) > 0:
        return f"⚠️ Project lies inside ESZ: {intersecting.iloc[0]['name']}"
    return "✅ No ESZ intersection."
```

This single feature would impress the professor more than anything else, because **it does what no LLM alone can do** — actual geospatial reasoning against authoritative datasets.

B. Regulatory Change Watch Agent (Agent 7)

A background daemon that runs daily:

- Polls bis.gov.in for new IS code amendments
- Polls irc.nic.in for new circulars
- Polls the Uttarakhand HC website for new mining-related orders
- When something new is found, re-checks all stored audits to see if any are now non-compliant
- Sends email alerts to project owners

This turns your **point-in-time audit** into a **continuous monitoring system**.

C. Multi-DPR Diff View

Two versions of the same DPR (V1 and V2 after revisions). Show a side-by-side comparison of which findings were addressed, which are new, which remain open. Critical for tracking audit responses across multiple submission cycles.

D. Direct Parivesh API Integration

If/when the Parivesh portal exposes an official API, integrate it directly. Submit clearance applications and pull status updates programmatically. This is the holy grail — closing the loop from "audit" to "actual government submission."

Priority Summary Cheat Sheet

#	Fix	Effort	Accuracy Gain	Do When
1	Remove <code>[:3500]</code> truncation	5 min	🔥🔥🔥	Right now
2	Temperature = 0 + seed	2 min	🔥🔥	Right now
3	Cross-agent context sharing	1 hour	🔥🔥	This weekend
4	Critic / verification agent	2 hours	🔥🔥🔥	This weekend
5	Pydantic structured output	3 hours	🔥🔥	This weekend
6	Vector knowledge base	1 day	🔥🔥	Next 2 weeks
7	Semantic PDF chunking	1 hour	🔥	Next 2 weeks
8	Table extraction (pdfplumber)	2 hours	🔥🔥	Next 2 weeks
9	Retry logic	1 hour	🔥	Next 2 weeks
10	Search caching	2 hours	🔥	Next 2 weeks
11	Confidence scoring	30 min	🔥🔥	Next 2 weeks
12	Parallel agent execution	4 hours	⚡ (speed)	Next month
13	Golden dataset	1 day	🔥🔥 (measurement)	Next month
14	PostgreSQL persistence	1 day	(no accuracy, but reliability)	Next month
15	PDF export	1 hour	(no accuracy, but UX)	Next month
A	Geospatial layer	1 week	🌟 wow factor	3 months
B	Regulatory watch agent	1 week	🌟 wow factor	3 months

The Single Most Important Insight

If you implement only ONE thing from this entire document, make it **Fix 4: the Critic Agent**.

Here's why: every other fix improves *how* you produce findings. The Critic Agent **catches the findings you produced wrongly**. It's the only fix that protects against hallucinations — which is the single biggest accuracy risk in any LLM system.

A flawed agent + a strong critic = trustworthy output. A perfect agent + no critic = a system you can never fully trust.



What to Say in the Viva About These Improvements

If the professor asks "What would you improve?", don't list 15 things — that signals scattered thinking. Pick three and rank them:

"Sir, three things in order. First, I'd add a critic agent that cross-examines every finding before showing it to the user — that catches hallucinations, which is the biggest accuracy risk in any LLM system. Second, I'd replace my flat JSON knowledge base with a FAISS vector store so only the most relevant rules reach the LLM instead of all of them — that improves focus. Third, I'd build a golden dataset of human-verified audits so I can measure accuracy quantitatively instead of just feeling that it's working."

That answer demonstrates: (a) you know what hallucination is, (b) you understand retrieval architecture, (c) you think about evaluation — three signs of an engineer who actually understands the field.

Good luck. 🚀